Core Design and MOOS Overview

Core MOOS-IvP design concepts and MOOS publish-subscribe context.

Included MIT MOOS-IvP PDF sources:

- chap_design.pdf
- chap_moos.pdf

Design Considerations of MOOS-IvP

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/chapters/chap_design`

Contents

1 Design Considerations of MOOS-IvP

The primary motivation in the design of MOOS-IvP is to build highly capable autonomous systems. Part of this picture includes doing so at a reduced short and long-term cost and a reduced time line. By ”design” we mean both the choice in architectures and algorithms as well as the choice to make key modules for infrastructure, basic autonomy and advanced tools available to the public under an Open Source license. The MOOS-IvP software design is based on three architecture philosophies, (a) the backseat driver paradigm, (b) publish and subscribe autonomy middleware, and (c) behavior based autonomy. The common thread is the ability to separate the development of software for an overall system into distinct modules coordinated by infrastructure software made available to the public domain.

1.1 Public Infrastructure - Layered Capabilities

The central architecture idea of both MOOS and IvP is the separation of overall capability into separate and distinct modules. The unique contributions of MOOS and IvP are the methods used to *coordinate* those modules. A second central idea is the decision to make algorithms and software modules for infrastructure, basic autonomy and advanced tools available to the public under an Open Source license. The idea is pictured in Figure 1. There are three things in this picture - (a) modules that actually perform a function (the wedges), (b) modules that coordinate other modules (the center of the wheel), and (c) standard wrapper software use by each module to allow it to be coordinated (the spokes).

The darker wedges in Figure 1 represent application modules (not infrastructure) that provide basic functionality and are publicly available. However, they do not hold any special immutable status. They can be replaced with a better version, or, since the source code is available, the code of the existing module can be changed or augmented to provide a better or different version (hopefully

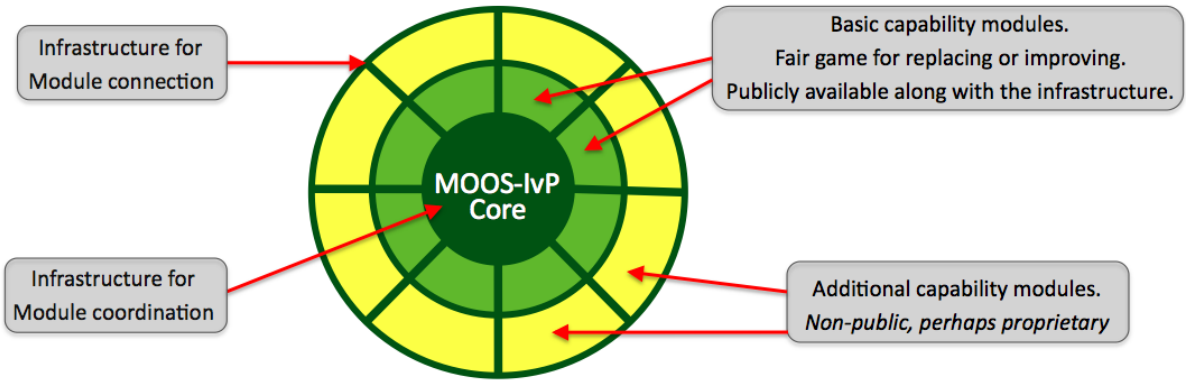

Figure 1: **Public Infrastructure - Layered Capabilities:** The center of the wheel represents MOOS-IvP Core. For MOOS this means the MOOSDB and the message passing and scheduling algorithms. For IvP this means the IvP helm behavior management and the multi-objective optimization solver. The wedges on the wheel represent individual modules - either MOOS processes or IvP behaviors. The spokes of the wheel represent the idea that each module inherits from a superclass to grab functionality key to plugging into the core. Each wedge or module contains a wrapper defined by the superclass that augments the function of the individual module. The darker wedges indicate publicly available modules and the lighter ones are modules added by users to augment the public set to comprise a particular fielded autonomy system.

with a different name - see the section on branching below). Later sections provide an overview of about 40 or so particular modules that are currently available. By modules we mean MOOS applications and IvP behaviors and the above comments hold in either case. The yellow wedges in Figure 1 represent the imaginable unimplemented modules or functionality. A particular fielded MOOS-IvP autonomy system typically is comprised of (a) the MOOS-IvP core modules, (b) *some* of the publicly available MOOS applications and IvP behaviors, and (c) additional perhaps non-public MOOS applications and IvP behaviors provided by one or more third party developers.

The objective of the public-infrastructure/layered-capabilities idea is to strike an important balance - the balance between effective code re-use and the need for users to retain privacy regarding how they choose to augment the public codebase with modules of their own to realize a particular autonomy system. The benefits of code re-use are an important motivation in fundamental architecture decisions in both MOOS and IvP. The modules that comprise the public MOOS-IvP codebase described in this document represent over twenty work-years of development effort. Furthermore, certain core components of the codebase have had hundreds if not thousands of hours of usage on a dozen or so fielded platform types in a variety of situations. The issue of code re-use is discussed next.

1.2 Code Re-Use

Code re-use is critical, and starts with the ability to have a system comprised of separate but coordinated modules. They key technical hurdle is to achieve module separation without invoking a substantial hit on performance. In short, MOOS middleware is a way of coordinating separate processes running on a single computer or over several networked computers. IvP is a way of coordinating several autonomy behaviors running within a single MOOS process.

Factors Contributing to Code Re-use:

- *Freedom from proprietary issues.* Software serving as infrastructure shared by all components (MOOS processes and IvP behaviors) are available under an Open Source license. In addition many mature MOOS and IvP modules providing commonly needed capabilities are also publicly available. Proprietary or non-publicly released code may certainly co-exist with non-proprietary public code to comprise a larger autonomy system. Such a system would retain a strategic edge over competitors if desired, but have a subset of components common with other users.

- *Module independence.* Maintaining or augmenting a system comprised of a set of distinct modules can begin to break down if modules are not independent with simple easy-to-augment interfaces. Compile dependencies between modules need to be minimized or eliminated. The maintenance of core software libraries and application code should be decoupled completely from the issues of 3rd party additional code.

- *Simple well-documented interfaces.* The effort required to add modules to the code base should be minimized. Documentation is needed for both (a) using the publicly available applications and libraries, and (b) guiding users in adding their own modules.

- *Freedom to innovate.* The infrastructure does not put undue restrictions on how basic problems can be solved. The infrastructure remains agnostic to techniques and algorithms used in the modules. No module is sacred and any module may be replaced.

Benefits of Code Re-Use:

- *Diversity of contributors.* Increasingly, an autonomy system contains many components that touch many areas of expertise. This would be true even for a vanilla use of a vehicle, but is compounded when considering the variety of sensors and missions and ways of exploiting sensors in achieving mission objectives. A system that allows for wide code re-use is also a system that allows module contributions from a wide set of developers or experts. This has a substantial impact on the issues mentioned below of lower cost, higher quality and reliability, and reduced development time line.

- *Lower cost.* One immediate benefit of code re-use is the avoidance of repeatedly re-inventing modules. A group can build capabilities incrementally and experts are free to concentrate on their area and develop only the modules that reflect their skill set and interests. Perhaps more important, code re-use gives the systems integrator *choices* in building a complete system from individual modules. Having choices leads to increased leverage in bargaining for favorable licensing terms or even non-proprietary terms for a new module. Favorable licensing terms arranged at the outset can lead to substantially lower long-term costs for future code maintenance or augmentation of software.

- *Higher performance capability.* Code re-use enhances performance capability in two ways. First, since experts are free to be experts without re-inventing the modules outside their expertise and provided by others, their own work is more likely to be more focused and efficient. They are likely to achieve a higher capability for a given a finite investment and given finite performance time. Second, since code re-use gives a systems integrator *choices*, this creates a meritocracy based on optimal performance-cost ratio of candidate software modules.

The under-capable, more expensive module is less likely to diminish the overall autonomy capability if an alternative module is developed to offer a competitive choice. Survival of the fittest.

- *Higher performance reliability.* An important part of system reliability is testing. The more testing time and the greater diversity of testing scenarios the better. And of course the more time spent testing on physical vehicles versus simulation the better. By making core components of a codebase public and permitting re-use by a community of users, that community provides back an enormous service by simply using the software and complaining when or if something goes wrong. Certain core components of the MOOS-IvP codebase have had hundreds if not thousands of hours of usage on a dozen or so platform types in a variety of situations. And many more hours in simulation. Testing doesn't replace good coding practice or formal methods for testing and verifying correctness, but it complements those two aspects and is enhanced by code re-use.

- *Reduced development time line.* Code re-use means less code is being re-developed which leads to quicker overall system development. More subtly, since code re-use can provide a systems integrator choices and competition on individual modules, development time can be reduced as a consequent. An integrator may simply accept the module developed the quickest, or the competition itself may speed up development. If choices and competition result in more favorable license agreements between the integrator and developer, this in itself may streamline agreements for code maintenance and augmentation in the long term. Finally, as discussed above, if code re-use leads to an element of community-driven bug testing, this will also quicken the pace in the evolution toward a mature and reliable autonomy system.

1.3 The Backseat Driver Design Philosophy

The key idea in the backseat driver paradigm is the separation between *vehicle control* and *vehicle autonomy*. The vehicle control system runs on a platform's main vehicle computer and the autonomy system runs on a separate payload computer. This separation is also referred to as the *mission controller - vehicle controller* interface. A primary benefit is the decoupling of the platform autonomy system from the actual vehicle hardware. The vehicle manufacturer provides a navigation and control system capable of streaming vehicle position and trajectory information from the main vehicle computer, and accepting a stream of autonomy decisions such as heading, speed and depth in return from the payload computer. Exactly how the vehicle navigates and implements control is largely unspecified to the autonomy system running in the payload. The relationship is depicted in Figure 2.

The autonomy system on the payload computer consists of a set of distinct processes communicating through a publish-subscribe database called the MOOSDB (Mission Oriented Operating Suite - Database). One such process is an interface to the main vehicle computer, and another key process is the IvP Helm implementing the behavior-based autonomy system. The MOOS community is referred to as the "larger autonomy" system, or the "autonomy system as a whole" since MOOS itself is middleware, and actual autonomous decision making, sensor processing, contact management etc., are implemented as individual MOOS processes.

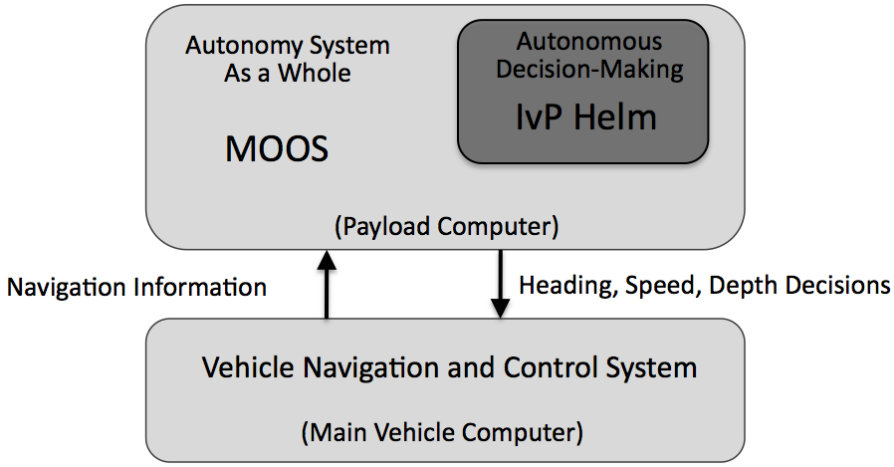

Figure 2: **The backseat driver paradigm**: The key idea is the separation of vehicle autonomy from vehicle control. The autonomy system provides heading, speed and depth commands to the vehicle control system. The vehicle control system executes the control and passes navigation information, e.g., position, heading and speed, to the autonomy system. The backseat paradigm is agnostic regarding how the autonomy system implemented, but in this figure the MOOS-IvP autonomy architecture is depicted.

1.4 The Publish-Subscribe Middleware Design Philosophy and MOOS

MOOS provides a middleware capability based on the publish-subscribe architecture and protocol. Each process communicates with each other through a single database process in a star topology (Figure 3). The interface of a particular process is described by what messages it produces (publications) and what messages it consumes (subscriptions). Each message is a simple variable-value pair where the values are typically either string or numerical values such as (`STATE`, `"DEPLOY"`), or (`NAV_SPEED`, `2.2`). MOOS messages may also contain raw binary data for passing images for example.

The key idea with respect to facilitating code re-use is that applications are largely independent, defined only by their interface, and any application is easily replaceable with an improved version with a matching interface. Since MOOS Core and many common applications are publicly available along with source code under an Open Source GPL license, a user may develop an improved module by altering existing source code and introduce a new version under a different name. With MOOS-IvP 13.2 which includes MOOS V10, the MOOS libraries are distributed under an LGPL license, to allow the development and use of commercial MOOS applications alongside open source applications. The MOOSDB and MOOS applications remain under an GPL license. The term MOOS Core refers to (a) the MOOSDB application, and (b) the MOOS Application superclass that each individual MOOS application inherits from to allow connectivity to a running MOOSDB. Holding the MOOS Core part of the codebase constant between MOOS developers enables the plug-and-play nature of applications.

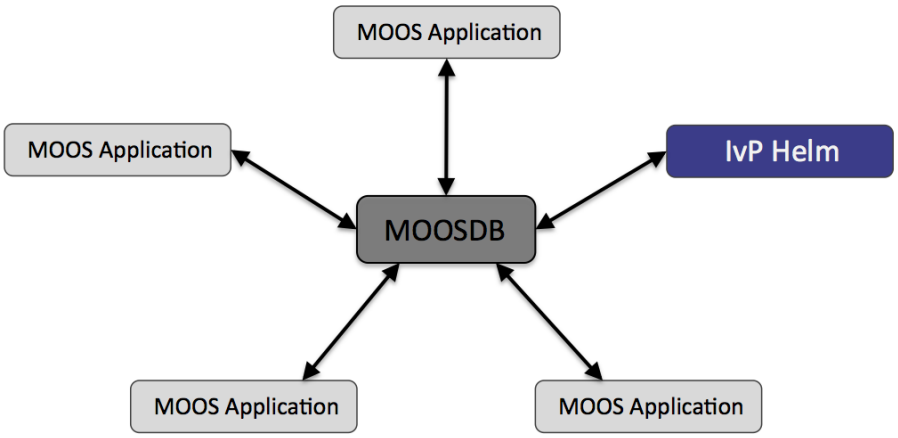

Figure 3: **A MOOS community**: is a collection of MOOS applications typically running on a single machine each with a separate process ID. Each process communicates through a single MOOS database process (the MOOSDB) in a publish-subscribe manner. Each process may be executing its inner-loop at a frequency independent from one another and set by the user. Processes may be all run on the same computer or distributed across a network.

1.5 The Behavior-Based Control Design Philosophy and IvP Helm

The IvP Helm runs as a single MOOS application and uses a behavior-based architecture for implementing autonomy. Behaviors are distinct software modules that can be described as self-contained mini expert systems dedicated to a particular aspect of overall vehicle autonomy. The helm implementation and each behavior implementation exposes an interface for configuration by the user for a particular set of missions. This configuration often contains particulars such as a certain set of waypoints, search area, vehicle speed, and so on. It also contains a specification of state spaces that determine which behaviors are active under what situations, and how states are transitioned. When multiple behaviors are active and competing for influence of the vehicle, the IvP solver is used to reconcile the behaviors (Figure 4).

The solver performs this coordination by soliciting an objective function, i.e., utility function, from each behavior defined over the vehicle decision space, e.g., possible settings for heading, speed and depth. In the IvP Helm, the objective functions are of a certain type - piecewise linearly defined - and are called IvP Functions. The solver algorithms exploit this construct to find a rapid solution to the optimization problem comprised of the weighted sum of contributing functions.

The concept of a behavior-based architecture is often attributed to [Brooks 1986]. Since then various solutions to the issue of action selection, i.e., the issue of coordinating competing behaviors, have been put forth and implemented in physical systems. The simplest approach is to prioritize behaviors in a way that the highest priority behavior locks out all others as in the Subsumption Architecture in [Brooks 1986]. Another approach is referred to as the potential fields, or vector summation approach (See [Arkin 1987], [Khatib 1985]) which considers the average action between multiple behaviors to be a reasonable compromise. These action-selection approaches have been used with reasonable effectiveness on a variety of platforms, including indoor robots, e.g., [Arkin 1987], [Arkin et al, 1993], [Pirjanian 1998], [Riekki 1999], land vehicles, e.g., [Rosenblatt 1997], and marine vehicles, e.g., [Bennett and Leonard 2000], [Carreras et al, 2000], [Kumar and Stover 2001,

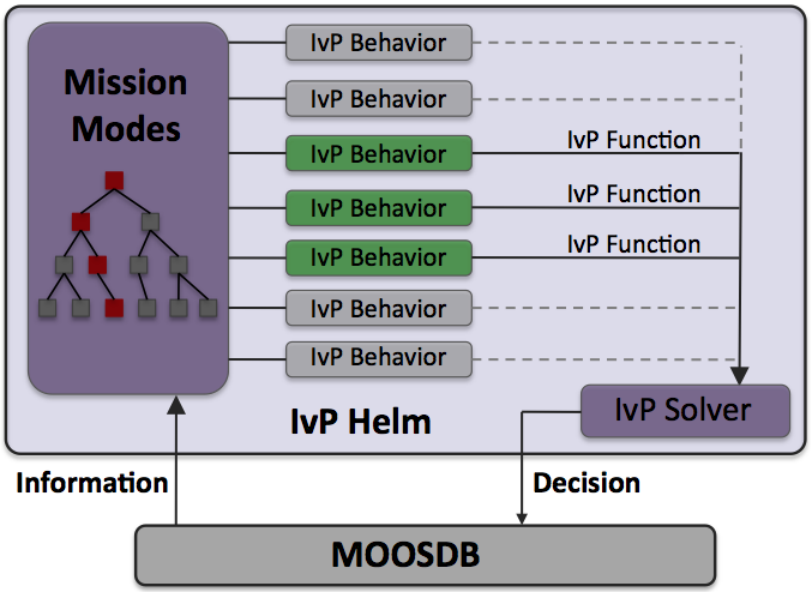

Figure 4: **The IvP Helm**: The helm is a single MOOS application running as the process pHelmIvP. It is a behavior-based architecture where the primary output of a behavior on each iteration is an IvP objective function. The IvP solver performs multi-objective optimization on the set of functions to find the single best vehicle action, which is then published to the MOOSDB. The functions are built and the set is solved on *each* iteration of the helm - typically one to four times per second. Only a subset of behaviors are active at any given time depending on the vehicle situation, and the state space configuration provided by the user.

[Rosenblatt et al, 2002], [Williams et al, 2000]. However, action-selection via the identification of a single highest priority behavior and via vector summation have well known shortcomings later described in [Pirjanian 1998], [Riekki 1999] and [Rosenblatt 1997] in which the authors advocated for the use of multi-objective optimization as a more suitable, although more computationally expensive, method for action selection. The IvP model is a method for implementing multi-objective function based action-selection that is computationally viable in the IvP Helm implementation.

References

- Paolo Pirjanian, `Multiple Objective Action Selection and Behavior Fusion`, PhD Thesis, Aalborg University, 1998.
- Rodney A. Brooks, `A Robust Layered Control System for a Mobile Robot`, IEEE Journal of Robotics and Automation, April 1986.
- George B. Dantzig, `Programming in a Linear Structure`, Comptroller, United States Air Force, February, 1948.
- Julio K. Rosenblatt, `DAMN: A Distributed Architecture for Mobile Navigation`, Journal of Experimental and Theoretical Artificial Intelligence, Volume 9, Number 2, 1997.
- Michael R. Benjamin, `The Interval Programming Model for Multi-Objective Decision Making`, Computer Science and Artificial Intelligence Laboratory, AIM-2004-021, MIT, Cambridge, MA.
- Jukka Riekki, `Reactive Task Execution of a Mobile Robot`, PhD Thesis, Oulu University, 1999.

- Ronald C. Arkin, Motor Schema Based Navigation for a Mobile Robot: An Approach to Programming by Behavior, Proceedings of the IEEE Conference on Robotics and Automation, March 1987.
- Oussama Khatib, Real-Time Obstacle Avoidance for Manipulators and Mobile Robots, Proceedings of the IEEE International Conference on Robotics and Automation, March 1985.
- Ronald C. Arkin and William M. Carter and Douglas C. Mackenzie, Active Avoidance: Escape and Dodging Behaviors for Reactive Control, International Journal of Pattern Recognition and Artificial Intelligence, 1993.
- Andrew A. Bennet and John J. Leonard, A Behavior-Based Approach to Adaptive Feature Detection and Following with Autonomous Underwater Vehicles, IEEE Journal of Oceanic Engineering, April, 2000.
- Marc Carreras and J. Batlle and Pere Ridao, Reactive Control of an AUV Using Motor Schemas, International Conference on Quality Control, Automation and Robotics, May, 2000.
- Ratnesh Kumar and James A. Stover, A Behavior-Based Intelligent Control Architecture with Application to Coordination of Multiple Underwater Vehicles, IEEE Transactions on Systems, Man, and Cybernetics - Part A: Cybernetics, November, 2001.
- Julio K. Rosenblatt and Stefan B. Williams and Hugh Durrant-Whyte, Behavior-Based Control for Autonomous Underwater Exploration, International Journal of Information Sciences, 145(1-2), 2002.
- Stefan B. Williams and Paul Newman and Gamini Dissanayake and Julio K. Rosenblatt and Hugh Durrant-Whyte, A decoupled, distributed AUV control architecture, Proceedings of 31st International Symposium on Robotics, Montreal, Canada, 2000.

A Very Brief Overview of MOOS

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/chapters/chap_moos`

Contents

1 A Very Brief Overview of MOOS

MOOS is often described as autonomy *middleware* which implies that it is a kind of glue that connects a collection of applications where the real work happens. MOOS does indeed connect a collection of applications, of which the IvP Helm is one. MOOS is cross platform stand-alone and dependency free. It needs no other third-party libraries. Each application inherits a generic MOOS interface whose implementation provides a powerful, easy-to-use means of communicating with other applications and controlling the relative frequency at which the application executes its primary set of functions. Due to its combination of ease-of-use, general extendibility and reliability, it has been used in the classroom by students with no prior experience, as well as on many extended field exercises with substantial robotic resources at stake. To frame the later discussion of the IvP Helm, the basic issues regarding MOOS applications are introduced here.

1.1 Inter-process communication with Publish/Subscribe

MOOS has a star-like topology as depicted in Figure **??**. Application within a MOOS community (a MOOSApp) have a connection to a single MOOS Database (called MOOSDB) lying at the heart of the software suite. All communication happens via this central server application. The network has the following properties:

- No peer-to-peer communication.
- Communication between the client and server is initiated by the client, i.e., the MOOSDB never makes a unsolicited attempt to contact a MOOSApp from out of the blue

- Each client has a unique name.
- A given client need have no knowledge of what other clients exist.
- One client does not transmit data to another - it can only be sent to the MOOSDB and from there to other clients. Modern versions of the library sport a sub-one millisecond latency when transporting multi-MB payloads between processes.
- The star network can be distributed over any number of machines running any combination of supported operating systems.
- The communications layer supports clock synchronization across all connected clients and in the same vein, can support ”time acceleration” whereby all connected clients operate in an accelerated time stream - something that is very useful in simulations involving many processes distributed over many machines.
- data can be sent in small bites as ”string” or ”double” packets or in arbitrarily large binary packets.

1.2 Message Content

The communications API in MOOS allows data to be transmitted between the MOOSDB and a client. The meaning of that data is dependent on the role of the client. However the form of that data is not constrained by MOOS although for the sake on convenience MOOS does offer bespoke support for small ”double” and string payloads. (Note that very early versions of MOOS only allowed data to be sent as strings or doubles - but this restriction is now long gone.) Data is packed into messages which contain other salient information shown in Table 1.

Variable	**Meaning**
Name	The name of the data
String Value	Data in string format
Double Value	Numeric double float data
Source	Name of client that sent this data to the `MOOSDB`
Auxiliary	Supplemental message information, e.g., IvP behavior source
Time	Time at which the data was written
Data Type	Type of data (`STRING` or `DOUBLE` or `BINARY`)
Message Type	Type of Message (usually `NOTIFICATION`)
Source Community	The community to which the source process belongs

Table 1: The contents of MOOS message.

Often it is convenient to send data in string format for example the string `"Type=EST,Name=AUV,Pos=[3x1]3.4,6.3,0.` might describe the position estimate of a vehicle called ”AUV” as a 3x1 column vector. This is human readable and does not require the sharing and synchronizing of header files to ensure both sender and recipient understand how to interpret data (as is the case with binary data). It is quite common for MOOS applications to communicate with string data in a concatenation of comma separated ”name=value” pairs.

- Strings are human readable.
- All data becomes the same type.

- Logging files are human readable (they can be compressed for storage).
- Replaying a log file is simply a case of reading strings from a file and ”throwing” them back at the MOOSDB in time order.
- The contents and internal order of strings transmitted by an application can be changed without the need to recompile consumers (subscribers to that data) - users simply would not understand new data fields but they would not crash.

The above are well understood benefits of sending self-explanatory ASCII data. However many applications use data types which do not lend themselves to verbose serialization to strings — think for example about camera image data being generated at 40Hz in full colour. At this point the need to send binary data is clear and of course MOOS supports it transparently (and the application pLogger supports logging and replaying it).

At this point it is up to the user to ensure that the binary data can be interpreted by all clients and that any and all perturbations to the data structures are distributed and compiled into each and every client. It is here that modern serialization tools such as ”Google Protocol Buffers” find application. They offer a seamless way to serialize complex data structures into binary streams. Crucially they offer *forward* compatibility – it is possible to update and augment data structures with new fields in the comforting knowledge that all existing apps will still be able to interpret the data - they just won't parse the new additions.

1.3 Mail Handling - Publish/Subscribe - in MOOS

Each MOOS application is a client having a connection to the MOOSDB. This connection is made on the client side and the client manages a threaded machinery that coordinates the communication with the MOOSDB. This completely hides the intricacies and timings of the communications from the rest of the application and provides a small, well defined set of methods to handle data transfer. The application can:

1. Publish data - issue a notification on named data.
2. Register for notifications on named data.
3. Collect notifications on named data - reading mail.

1.3.1 Publishing Data

Data is published as a pair - a variable and value - that constitute the heart of a MOOS message described in Table 1. The client invokes the `Notify(VarName, VarValue)` command where appropriate in the client code. The above command is implemented both for string, double and binary values, and the rest of the fields described in Table 1 are filled in automatically. Each notification results in another entry in the client's ”outbox”, which in older versions of MOOS, is emptied the next time the `MOOSDB` accepts an incoming call from the client or in recent versions, is pushed instantaneously to all interested clients.

1.3.2 Registering for Notifications

Assume that a list of names of data published has been provided by the author of a particular MOOS application. For example, a application that interfaces to a GPS sensor may publish data called GPS_X and GPS_Y. A different application may register its interest in this data by subscribing or registering for it. An application can register for notifications using a single method `Register()` specifying both the name of the data and the maximum rate at which the client would like to be informed that the data has been changed. The latter parameter is specified in terms of the minimum possible time between notifications for a named variable. For example setting it to zero would result in the client receiving each and every change notification issued on that variable. MOOS V10 and later also supports "wildcard" subscriptions. For example a client can register for `"*:*"` to receive all messages from all other clients. Or `"GPS_*:?NAV"` to receive messages beginning with `"GPS_"` from any process with a four letter name ending in `"NAV"`.

1.3.3 Reading Mail

A client can enquire at any time whether it has received any new notifications from the MOOSDB by invoking the `Fetch` method. The function fills in a list of notification messages with the fields given in Table 1. Note that a single call to `Fetch` may result in being presented with several notifications corresponding to the same named data. This implies that several changes were made to the data since the last client-server conversation. However, the time difference between these similar messages will never be less than that specified in the `Register()` function described above. In typical applications the `Fetch` command is called on the client's behalf just prior to the `Iterate()` method, and the messages are handled in the user overloaded `OnNewMail()` method.

1.4 Overloaded Functions in MOOS Applications

MOOS provides a base class called `CMOOSApp` which simplifies the writing of a new MOOS application as a derived subclass. Beneath the hood of the `CMOOSApp` class is a loop which repetitively calls a function called `Iterate()` which by default does nothing. One of the jobs as a writer of a new MOOS-enabled application is to flesh this function out with the code that makes the application do what we want. Behind the scenes this overall loop in `CMOOSApp` is also checking to see if new data has been delivered to the application. If it has, another virtual function, `OnNewMail()`, is called. This is the function within which code is written to process the newly delivered data.

The roles of the three virtual functions in Figure 1 are discussed below. The pHelmIvP application does indeed inherit from `CMOOSApp` and overload these functions. The base class contains other virtual functions (`OnConnectToServer()` and `OnDisconnectFromServer()`).

1.4.1 The Iterate() Method

By overriding the `CMOOSApp::Iterate()` function in a new derived class, the author creates a function from which the work that the application is tasked with doing can be orchestrated. In the pHelmIvP application, this method will consider the next best vehicle decision, typically in the form of deciding values for the vehicle heading, speed and depth. The rate at which `Iterate()` is called by the `SetAppFreq()` method or by specifying the AppTick parameter in a mission file. Note that the requested frequency specifies the maximum frequency at which `Iterate()` will be called - it does not

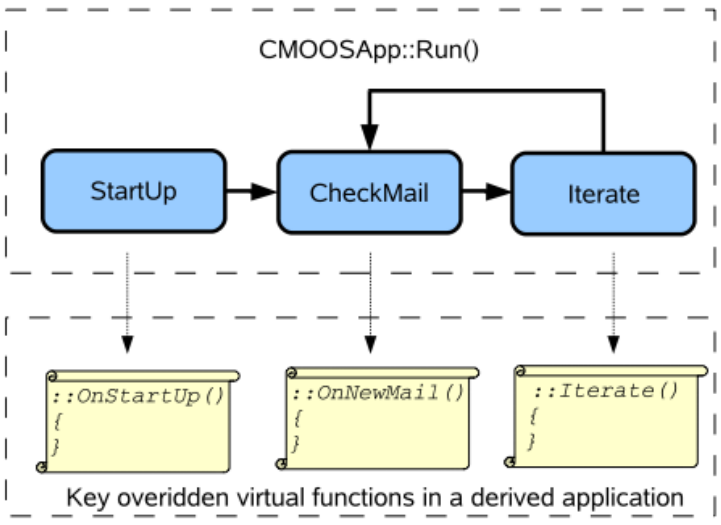

Figure 1: **Key virtual functions of the MOOS application base class**: The flow of execution once `Run()` has been called on a class derived from `CMOOSApp` . The scrolls indicate where users of the functionality of CMOOSApp will be writing new code that implements whatever it is that is wanted from the new applications. Note that it is not the case (as the above may suggest) that mail is polled for - in modern incantations of MOOS it is pushed to a client a synchronously `OnNewMail()` is called as soon as `Iterate()` is not running.

guarantee that it will be called at the requested rate. For example if you write code in `Iterate()` that takes 1 second to complete there is no way that this method can be called at more than 1Hz. If you want to call `Iterate()` as fast as is possible simply request a frequency of zero - but you may want to reconsider why you need such a greedy application.

1.4.2 The OnNewMail() Method

Just before `Iterate()` is called, the `CMOOSApp` base class determines whether new mail is present, i.e., whether some other process has posted data for which the client has previously registered, as described above. If new mail is waiting, the `CMOOSApp` base class calls the `OnNewMail()` virtual function, typically overloaded by the application. The mail arrives in the form of a list of `CMOOSMsg` objects (see Table 1). The programmer is free to iterate over this collection examining who sent the data, what it pertains to, how old it is, whether or not it is string or numerical data and to act on or process the data accordingly. In recent versions of MOOS it is possible to have `OnNewMail()` called in a directly and rapidly in response to new mail being received by the back-end communications threads. This architecture allows for very rapid response times (sub ms) between a client posting data and it being received and handled by all interested parties.

1.4.3 The OnStartUp() Method

The `OnStartUp()` function is called just before the application enters into its own forever-loop depicted in Figure 1. This is the application that implements the application's initialization code, and in particular reads configuration parameters (including those that modify the default behavior of the CMOOSApp base class) from a file.

1.5 MOOS Mission Configuration Files

Every MOOS process can read configuration parameters from a mission file which by convention has a `.moos` extension. Traditionally MOOS processes share the same mission file to the maximum extent possible. For example, it is customary for there to be one common mission file for all MOOS processes running on a given machine. Every MOOS process has information contained in a configuration block within a `*.moos` file. The block begins with the statement

```
ProcessConfig = ProcessName
```

where `ProcessName` is the unique name the application will use when connecting to the MOOSDB. The configuration block is delimited by braces. Within the braces there is a collection of parameter statements, one per line. Each statement is written as:

```
ParameterName = value
```

where `value` can be any string or numeric value. All applications deriving from `CMOOSApp` inherit several important configuration options. The most important options for `CMOOSApp` derived applications are `CommsTick` and `AppTick`. The former configures how often the communications thread talks to the `MOOSDB` and the latter how often (approximately) `Iterate()` will be called.

Parameters may also be defined at the "global" level, i.e., not in any particular process' configuration block. Three parameters that are mandatory and typically found at the top of all `*.moos` files are: `ServerHost` naming the IP address associated with the MOOSDB server being launched with this file, `ServerPort` naming the port number over which the MOOSDB server is communicating with clients, and `Community` naming the community comprising the server and clients. An example is shown in lines 1-3 in Listing 4.

1.6 Launching Groups of MOOS Applications with Antler

Antler provides a simple and compact way to start a MOOS mission comprised of several MOOS processes, a.k.a., a MOOS *community*. For example if the desired mission file is `alpha.moos` then executing the following from a terminal shell:

```
$ cd moos-ivp/ivp/missions/s1_alpha
$ pAntler alpha.moos
```

will launch the required processes for the mission. It reads from its configuration block (which is declared as `ProcessConfig=ANTLER`) a list of process names that will constitute the MOOS community. Each process to be launched is specified with a line with the general syntax

where `LaunchConfiguration` is an optional comma-separated list of `parameter=value` pairs which collectively control how the process `procname` (for example pHelmIvP, or pLogger or MOOSDB) is launched. Exactly what parameters can be specified is outside the scope of this discussion. Antler looks through its entire configuration block and launches one process for every line which begins with the `RUN=` left-hand side. When all processes have been launched Antler waits for all of them to exit and then quits itself.

There are many more aspects of Antler not discussed here but can be found in the Antler documentation at the MOOS web site (see Section **??**). These include hooks for altering the console appearance for each launched process, controlling the search path for specifying how executables are located on the host file system, passing parameters to launched processes, running multiple instances of a particular process, and using Antler to launch multiple distinct communities on a network.

1.7 Scoping and Poking the MOOSDB

An important tool for writing and debugging MOOS applications (and IvP Helm behaviors) is the ability for the user to interact with an active MOOS community and see the current values of particular MOOS variables (scoping the DB) and to alter one or more variables with a desired value (poking the DB). Below are listed tools for scoping and poking respectively. More information on each can be found on the Oxford or MIT web sites, or in in some instances, other parts of this document.

Tools for scoping the MOOSDB:

- uMS - A GUI-based tool written in FLTK and maintained and distributed from the Oxford website.
- uXMS - A terminal-based tool maintained and distributed from the MIT website
- uHelmScope - A terminal-based tool specialized for displaying information about a running instance of the helm, but it also contains a general-purpose scoping utility similar to uXMS. Distributed from the MIT website.

Tools for poking the MOOSDB:

- uMS - The GUI-based tool for scoping, listed above, also provides a means for poking. Distributed from the Oxford website.
- uPokeDB - A light-weight command-line tool for poking one or more variable-value pairs, with the option of scoping on the before and after values of the poked variable before exiting. Distributed from the MIT website.
- pMarineViewer - A GUI-based tool primarily used for rendering the paths of vehicles in 2D space on a Geo display, but also can be configured to poke the DB with variable-value pairs connected to buttons on the display. Distributed from the MIT website.
- uTimerScript - Allows the user to script a set of pre-configured pokes to a MOOSDB with each entry in the script happening after a speci- fied amount of time. Script may be paused or fast-forwarded. Events may also be configured with random values and happen randomly in a chosen window of time. Distributed from the MIT website.

- uTermCommand - A terminal-based tool for poking the DB with pre-defined variable-value pairs. The user can configure the tool to associate aliases (as short as a single character) to quickly poke the DB. Distributed from the MIT website.
- iRemote - A terminal-based tool for remote control of a robotic platform running MOOS. It can be configured to associate a pre-defined variable-value poke with any un-mapped key on the keyboard. Distributed from the Oxford website.

The above list is almost certainly not a complete list for scoping and poking a MOOSDB, but it's a decent start.

1.8 A Simple MOOS Application - pXRelay

The bundle of applications distributed from `www.moos-ivp.org` contains a very simple MOOS application called pXRelay. The pXRelay application registers for a single "input" MOOS variable and publishes a single "output" MOOS variable. It makes a single publication on the output variable for each mail message received on the input variable. The value published is simply a counter representing the number of times the variable has been published. By running two (differently named) versions of pXRelay with complementary input/output variables, the two processes will perpetuate some basic publish/subscribe handshaking. This application is distributed primarily as a simple example of a MOOS application that allows for some illustration of the following topics introduced up to this point:

- Finding and launching with pAntler example code distributed with the MOOS-IvP software bundle.
- An example mission configuration file.
- Scoping variables on a running MOOSDB with the uXMS tool.
- Poking the MOOSDB with variable-value pairs using the uPokeDB tool.
- Illustrating the OnStartUp(), OnNewMail(), and Iterate() overloaded functions of the `CMOOSApp` base class.

Besides touching on these topics, the collection of files in the pXRelay source code sub-directory is not a bad template from which to build your own modules.

1.8.1 Finding and Launching the pXRelay Example

The pXRelay example mission should be in the same directory tree containing the source code. There is a single mission file, `xrelay.moos`:

```
moos-ivp/
   MOOS/
   ivp/
      missions/
         xrelay/
            xrelay.moos    <---- The MOOS file
```

To run this mission from a terminal window, simply change directories and launch:

```
$ cd moos-ivp/ivp/missions/xrelay
$ pAntler xrelay.moos
```

After pAntler has launched each process, there should be four open terminal windows, one for each pXRelay process, one for uXMS, and one for the MOOSDB itself.

1.8.2 Scoping the pXRelay Example with uXMS

Among the four windows launched in the example, the window to watch is the uXMS window, which should have output similar to the following (minus the line numbers):

Listing 1.1: Example uXMS output after the pXRelay example is launched.

```
0   VarName            (S)ource       (T)ime     (C)ommunity  VarValue
1   ----------------   ----------     ---------  ----------   ----------- (73)
2   APPLES              n/a             n/a       n/a           n/a
3   PEARS               n/a             n/a       n/a           n/a
4   APPLES_ITER_HZ      pXRelay_APPLES 14.93     xrelay       24.93561
5   PEARS_ITER_HZ       pXRelay_PEARS  14.94     xrelay       24.93683
6   APPLES_POST_HZ      n/a             n/a       n/a           n/a
7   PEARS_POST_HZ       n/a             n/a       n/a           n/a
```

Initially the only thing that is changing in this window is the integer at the end of line 1 representing the number of updates written to the terminal. Here uXMS is configured to scope on the six variables shown in the VarName column. Column 2 shows which process last posted on the variable, column 3 shows when the last posting occurred, column 4 shows the community name from which the post originated, and column 5 shows the current value of the variable. The "n/a" entries indicate that a process has yet to write to the given variable.

There are two pXRelay processes running - one under the alias pXRelay_APPLES publishing the variable APPLES as its output variable, APPLES_ITER_HZ indicating the frequency in which the Iterate() function is executed, and APPLES_POST_HZ indicating the frequency at which the output variable is posted. There is likewise a pXRelay_PEARS process and the corresponding output variables.

1.8.3 Seeding the pXRelay Example with the uPokeDB Tool

Upon launching the pXRelay example, the only variables actively changing are the *_ITER_HZ variables (lines 4-5 in Listing 1) which confirm that the Iterate() loop in each process is indeed being executed. The output for the other variables in Listing 1 reflect the fact that the two processes have not yet begun handshaking. This can be kicked off by poking the APPLES (or PEARS) variable, which is the input variable for pXRelay_PEARS, by typing the following:

```
$ cd moos-ivp/ivp/missions/xrelay
$ uPokeDB xrelay.moos APPLES=1
```

The uPokeDB tool will publish to the MOOSDB the given variable-value pair APPLES=1. It also takes as an argument the mission file, xrelay.moos, to read information on where the MOOSDB is running in

terms of machine name and port number. The output should look similar to the following:

Listing 1.2: Example uPokeDB output after poking the MOOSDB with APPLES=1.

```
0  PRIOR to Poking the MOOSDB
1    VarName               (S)ource      (T)ime       VarValue
2    ----------------      ----------    ----------   -------------
3    APPLES
4
5
6  AFTER Poking the MOOSDB
7    VarName               (S)ource      (T)ime       VarValue
8    ----------------      ----------    ----------   -------------
9    APPLES                uPokeDB       40.19        1.00000"
```

The output of uPokeDB first shows the value of the variable prior to the poke, and then the value afterwards. Once the MOOSDB has been poked as above, the pXRelay_PEARS application will receive this mail and, in return, will write to its output variable PEARS, which in turn will be read by pXRelay_APPLES and the two processes will continue thereafter to write and read their input and output variables. This progression can be observed in the uXMS terminal, which may look something like that shown in Listing 3:

Listing 1.3: Example uXMS output after the pXRelay example is seeded.

```
0   VarName           (S)ource         (T)ime    (C)ommunity   VarValue
1   ----------------   ----------      --------  ----------    ---------- (221)
2   APPLES             pXRelay_APPLES  44.78     xrelay        151
3   PEARS              pXRelay_PEARS   44.74     xrelay        151
4   APPLES_ITER_HZ     pXRelay_APPLES  44.7      xrelay        24.90495
5   PEARS_ITER_HZ      pXRelay_PEARS   44.7      xrelay        24.90427
6   APPLES_POST_HZ     pXRelay_APPLES  44.79     xrelay        8.36411
7   PEARS_POST_HZ      pXRelay_PEARS   44.74     xrelay        8.36406
```

Upon each write to the MOOSDB the value of the variable is incremented by 1, and the integer progression can be monitored in the last column on lines 2-3. The APPLES_POST_HZ and PEARS_POST_HZ variables represent the frequency at which the process makes a post to the MOOSDB. This of course is different than (but bounded above by) the frequency of the Iterate() loop since a post is made within the Iterate() loop only if mail had been received prior to the outset of the loop. In a world with no latency, one might expect the "post" frequency to be exactly half of the "iterate" frequency. We would expect the frequency reported on lines 6-7 to be no greater than 12.5, and in this case values of about 8.4 are observed instead.

1.8.4 The pXRelay Example MOOS Configuration File

The mission file used for the pXRelay example, `xrelay.moos` is discussed here. This file is provided as part of the MOOS-IvP software bundle under the "missions" directory as discussed above in Section 1.8.1. It is discussed here in three parts in Listings 4 through 6 below.

The part of the `xrelay.moos` file provides three mandatory pieces of information needed by the MOOSDB process for launching. The MOOSDB is a server and on line 1 is the IP address for the machine, and line 2 indicates the port number where clients can expect to find the MOOSDB once it has been launched. Since each MOOSDB and the set of connected clients form a MOOS "community", the

community name is provided on line 3. Note the `xrelay` community name in the `xrelay.moos` file and the community name in column 4 of the uXMS output in Listing 1 above.

Listing 1.4: The `xrelay.moos` *mission file for the pXRelay example.*

```
1  ServerHost = localhost
2  ServerPort = 9000
3  Community  = xrelay
4
5  //------------------------------------------
6  // Antler configuration  block
7  ProcessConfig = ANTLER
8  {
9    MSBetweenLaunches = 200
10
11   Run = MOOSDB  @ NewConsole = true
12   Run = pXRelay  @ NewConsole = true ~ pXRelay_PEARS
13   Run = pXRelay  @ NewConsole = true ~ pXRelay_APPLES
14   Run = uXMS  @ NewConsole = true
15 }
```

The configuration block in lines 7-15 of `xrelay.moos` is read by the pAntler for launching the processes or clients of the MOOS community. Line 9 specifies how much time, in milliseconds, between the launching of processes. Lines 11-14 name the four MOOS applications launched in this example. On these lines, the component `"NewConsole = true"` determines whether a new console window will be opened for each process. Try changing them to `false` - only the uXMS window really needs to be open. The others merely provide a visual confirmation that a process has been launched. The "`~ pXRelay_PEARS`" component of lines 12 and 13 tell pAntler to launch these applications with the given alias. This is required here since each MOOS client needs to have a unique name, and in this example two instances of the pXRelay process are being launched.

In lines 17-39 in Listing 5-B below, the two pXRelay applications are configured. Note that the argument to `ProcessConfig` on lines 20 and 32 is the alias for pXRelay specified in the Antler configuration block on lines 12 and 13. Each pXRelay process is configured such that its incoming and outgoing MOOS variables complement one another on lines 25-26 and 37-38. Note the AppTick parameter (see Section 1.4.1) is set to 25 in both configuration blocks, and compare with the observed frequency of the Iterate() function reported in the variables APPLES_ITER_HZ and PEARS_ITER_HZ in Listing 1. MOOS has done a pretty faithful job in this example of honoring the requested frequency of the Iterate() loop in each application.

Listing 1.5: The xrelay.moos mission file - configuring the pXRelay processes.

```
17 //------------------------------------------
18 // pXRelay config block
19
20 ProcessConfig = pXRelay_APPLES
21 {
22   AppTick       = 25
23   CommsTick     = 25
24
25   OUTGOING_VAR  = APPLES
26   INCOMING_VAR  = PEARS
27 }
28
29 //------------------------------------------
30 // pXRelay config block
31
```

```
32  ProcessConfig = pXRelay_PEARS
33  {
34    AppTick       = 25
35    CommsTick     = 25
36
37    INCOMING_VAR  = APPLES
38    OUTGOING_VAR  = PEARS
39  }
```

In the last portion of the `xrelay.moos` file, shown in Listing 6-C below, the uXMS process is configured. In this example, uXMS is configured to scope on the six variables specified on lines 54-59 to give the output shown in Listings 1 and 3. By setting the `paused` parameter on line 49 to `false`, the output of uXMS is continuously and automatically updated - in this case four times per second due to the rate of 4Hz specified in lines 46-47. The `display_*` parameters in lines 50-52 ensure that the output in columns 2-4 of the uXMS output is expanded.

Listing 1.6: Configuring uXMS in the pXRelay example.

```
41  //------------------------------------------
42  // uXMS config block
43
44  ProcessConfig = uXMS
45  {
46    AppTick    = 4
47    CommsTick  = 4
48
49    paused            = false
50    display_source    = true
51    display_time      = true
52    display_community = true
53
54    var  = APPLES
55    var  = PEARS
56    var  = APPLES_ITER_HZ
57    var  = PEARS_ITER_HZ
58    var  = APPLES_POST_HZ
59    var  = PEARS_POST_HZ
60  }
```

1.8.5 Suggestions for Further Things to Try with this Example

- Take a look at the `OnStartUp()` method in the `XRelay.cpp` class in the pXRelay module in the software bundle to see how the handling of parameters in the `xrelay.moos` configuration file are implemented, and the subscription for a MOOS variable.
- Take a look at the `OnNewMail()` method in the `XRelay.cpp` class in the pXRelay module in the software bundle to see how incoming mail is parsed and handled.
- Take a look at the `Iterate()` method in the `XRelay.cpp` class in the pXRelay module in the software bundle to see an example of a MOOS process that acts upon incoming mail and conditionally posts to the MOOSDB
- Try changing the `AppTick` parameter in *one* of the pXRelay configuration blocks in the `xrelay.moos` file, re-start, and note the resulting change in the iteration and post frequencies in the uXMS output.

- Try changing the CommsTick parameter in *one* of the pXRelay configuration blocks in the `xrelay.moos` file to something much lower than the AppTick parameter, re-start, and note the resulting change in the iteration and post frequencies in the uXMS output.

1.9 MOOS Applications Available to the Public

Below are very brief descriptions of MOOS applications in the public domain. This is by no means a complete list. It does not include applications outside MIT and Oxford, and it is not even a complete list of applications from those organizations.

1.9.1 MOOS Modules from Oxford

- pAntler: A tool for launching a collection of MOOS processes given a mission file.
- pShare: A tool that allows messages to pass between communities and allows for the renaming of messages as they are shuffled between communities.
- pLogger: A logger for recording the activities of a MOOS session. It can be configured to record a fraction of, or all publications of any number of MOOS variables.
- uMS: A GUI-Based MOOS scope for monitoring one or more MOOSDBs.
- uPlayback: An FLTK-based, cross platform GUI application that can load in log files and replay them into a MOOS community as though the originators of the data were really running and issuing notifications.
- iMatlab: An application that allows Matlab to join a MOOS community - even if only for listening in and rendering sensor data. It allows connection to the MOOSDB and access to local serial ports.
- iRemote: A terminal-based tool for remote control of a robotic platform running MOOS. It can be configured to associate a pre-defined variable-value poke with any un-mapped key on the keyboard.

1.9.2 Mission Monitoring Modules

Mission monitoring modules aid the user in either keeping a high-level tab on the mission as it unfolds, or help the user analyze and debug a mission. In release 13.2 this includes two powerful new tools for appcast monitoring, uMAC and uMACView. The pMarineViewer has also been substantially augmented to support appcast viewing.

- pMarineViewer: GUI tool for rendering events in an area of vehicle operation. It repeatedly updates vehicle positions from incoming node reports, and will render several geometric types published from other MOOS apps. The viewer may also post messages to the MOOSDB based on user-configured keyboard or mouse events. See the online documentation for pMarineViewer.
- uHelmScope: A terminal-based (non-GUI) scope onto a running IvP Helm process, and key MOOS variables. It provides behavior summaries, activity states, and recent behavior postings to the MOOSDB. A very useful tool for debugging helm anomalies. See the documenation for uHelmScope.
- uXMS: A terminal-based (non GUI) tool for scoping a MOOSDB Users may precisely configure the set of variables they wish to scope on by naming them explicitly on the command line or

in the MOOS configuration block. The variable set may also be configured by naming one or more MOOS proceses on which all variables published by those processes will be scoped. Users may also scope on the *history* of a single variable. See the documentation for uXMS.

- uProcessWatch: This application monitors the presence of MOOS apps on a watch-list. If one or more are noted to be absent, it will be so noted on the MOOS variable PROC_WATCH_SUMMARY. uProcessWatch is appcast-enabled and will produce a succinct table summary of watched processes and the CPU load reported by the processes themselves. The items on the watch list may be named explicitly in the config file or inferred from the Antler block or from list of DB_CLIENTS. An application may be excluded from the watch list if desired. See the documentation for uProcessWatch.
- uMAC: The uMAC application is a utility for Monitoring AppCasts. It is launched and run in a terminal window and will parse appcasts generated within its own MOOS community or those from other MOOS communities bridged or shared to the local MOOSDB. The primary advantage of uMAC versus other appcast monitoring tools is that a user can remotely log into a vehicle via ssh and launch uMAC locally in a terminal. See the documentation for uMAC.
- uMACView: A GUI tool for visually monitoring appcasts. It will parse appcasts generated within its own MOOS community or those from other MOOS communities bridged or shared to the local MOOSDB. Its capability is nearly identical to the appcast viewing capability built into pMarineViewer. It was intended to be an appcast viewer for non-pMarineViewer users. See the documentation for uMACView.

1.9.3 Mission Execution Modules

Mission execution modules participate directly in the proper execution of the mission rather than simply helping to monitor, plan or analyze the mission.

- pNodeReporter: A tool for collecting node information such as present vehicle position, trajectory and type, and posting it in a single report for sharing between vehicles or sending to a shoreside display. See the documentation for pNodeReporter.
- pContactMgrV20: The contact manager deals with other known vehicles in its vicinity. It handles incoming reports perhaps received via a sensor application or over a communications link. Minimally it posts summary reports to the MOOSDB, but may also be configured to post alerts with user-configured content about one or more of the contacts. May be used in conjunction with the helm to spawn contact-related behaviors for collision avoidance, tracking, etc. See the documentation for pContactMgrV20.
- pEchoVar: A tool for subscribing for a variable and re-publishing it under a different name. It also may be used to pull out certain fields in string publications consisting of comma-separated parameter=value pairs, publishing the new string using different parameters. See the documentation for pEchoVar.
- pSearchGrid: An application for storing a history of vehicle positions in a 2D grid defined over a region of operation.

1.9.4 Mission Simulation Modules

Mission simulation modules are used only in simulation. Many of the applications in the uField Toolbox may also be considered simulation modules, but they also have a use case involving simulated sensors on actual physical vehicles. The two modules below are purely for simulated vehicles.

- uSimMarineV22: A simple 3D vehicle simulator that updates vehicle state, position and trajectory, based on the present actuator values and prior vehicle state. Typical usage scenario has a single instance of uSimMarineV22 associated with each simulated vehicle. See the documentation for uSimMarineV22.
- uSimCurrent: A simple application for simulating the effects of water current. Based on local current information from a given file, it repeately reads the vehicle's present position and publishes a drift vector, presumably consumed by uSimMarine.

1.9.5 Modules for Poking the MOOSDB

Poking the MOOSDB is a common and often essential part of mission execution and/or command and control. The pMarineViewer tool also contains several methods for poking the MOOSDB on user command.

- uPokeDB: A command-line tool for poking a MOOSDB with variable-value pairs provided on the command line. It finds the MOOSDB via mission file provided on the command line, or the IP address and port number given on the command line. It will connect to the DB, show the value prior to poking, poke the DB, and wait for mail from the DB to confirm the result of the poke. See the documentation for uPokeDB.
- uTimerScript: Allows the user to script a set of pre-configured pokes to a MOOSDB with each entry in the script happening after a speci- fied amount of time. Script may be paused or fast-forwarded. Events may also be configured with random values and happen randomly in a chosen window of time. See the documentation for uTimerScript.
- uTermCommand: A terminal application for poking the MOOSDB with pre-defined variable-value pairs. A unique key may be associated with each poke. See the documentation for uTermCommand.

1.9.6 The Alog Toolbox

The Alog Toolbox is set of offline tools for analyzing and manipulating alog files produces by the pLogger application distributed with the Oxford MOOS codebase.

- alogscan: A command line tool for reporting the contents of a given MOOS .alog file. See the documentation for alogscan, part of the Alog Toolbox.
- alogclip: A command line tool that will create a new MOOS .alog file from a given .alog file by removing entries outside a given time window. See the documentation for alogclip, part of the Alog Toolbox.
- aloggrep: A command line tool that will create a new MOOS .alog file by retaining only the given MOOS variables or sources from a given .alog file. See the documentation for aloggrep, part of the Alog Toolbox.

- alogrm: A command line tool that will create a new MOOS .alog file by removing the given MOOS variables or sources from a given .alog file. See the documentation for alogrm, part of the Alog Toolbox.
- alogview: A GUI tool for analyzing a vehicle mission by plotting one or more vehicle trajectories on the operation area, while viewing a plot of any of the numerical values in the alog file(s). See the documentation for alogview, part of the Alog Toolbox.

1.9.7 The uField Toolbox

The uField Toolbox contains a number of tools for supporting multi-vehicle missions where each vehicle is connected to a shoreside community. This includes both simulation and real field experiments. It also contains a number of simulated sensors that run on offboard the vehicle on the shoreside.

- pHostInfo: Automatically detect the vehicle's host information including the IP addresses, port being used by the MOOSDB, the port being used by local pShare for UDP listening, and the community name for the local MOOSDB. Post these to facilitate automatic intervehicle communications in especially in multi-vehicle scenarios where the local IP address changes with DHCP.
- uFldNodeBroker: Typically run on a vehicle or simulated vehicle in a multi-vehicle context. Used for making a connection to a shoreside community by sending local information about the vehicle such as the IP address, community name, and port number being used by pShare for incoming UDP messages. Presumably the shoreside community uses this to know where to send outgoing UDP messages to the vehicle. See the documentation for uFldNodeBroker, part of the uField Toolbox.
- uFldShoreBroker: Typically run in a shoreside community. Takes reports from remote vehicles describing how they may be reached. Posts registration requests to shoreside pShare to bridge user-provided list of variables out to vehicles. Upon learning of vehicle JAKE will create bridges FOO_ALL and FOO_JAKE to JAKE, for all such user-configured variables. See the documentation for uFldShoreBroker, part of the uField Toolbox.
- uFldNodeComms: A shoreside tool for managing communications between vehicles. It has knowledge of all vehicle positions based on incoming node reports. Communications may be limited based on vehicle range, frequency of messages, or size of message. Messages may also be blocked based on a team affiliation. See the documentation for uFldNodeComms, part of the uField Toolbox.
- uFldMessageHandler: A tool for handling incoming messages from other nodes. The message is a string that contains the source and destination of the message as well as the MOOS variable and value. This app simply posts to the local MOOSDB the variable-value pair contents of the message. See the documentation for uFldMessageHandler, part of the uField Toolbox.
- uFldScope: Typically run in a shoreside community. Takes information from user-configured set of incoming reports and parses out key information into a concise table format. Reports may be any report in the form of comma-separated parameter-value pairs.
- uFldPathCheck: Typically run in a shoreside community. Takes node reports from remote vehicles and calculates the current vehicle speed and total distance travelled and posts them

in two concise reports. Odometry tallies may be re-set to zero by other apps. See the documentation for uFldPathCheck, part of the uField Toolbox.

- uFldHazardSensor: Typically run in a shoreside community. Configured with a set objects with a given x,y location and classification (hazard or beningn). The sensor simulator receives a series of requests from a remote vehicle. When sensor determines that an object is is within the sensor field of a requesting vehicle, it may or may not return a sensor detection report for the object, and perhaps also a proper classification. The odds of receiving a detection and proper classification depend on the sensor configuration and the user's preference for P_D/P_FA on the prevailing ROC curve.
- uFldHazardMetric: An application for grading incoming hazard reports, presumably generated by users of the uFldHazardSensor after exploring a simulated hazard field.
- uFldHazardMgr: The uFldHazardMgr is a strawman MOOS app for managing hazard sensor information and generation of a hazard report over the course of an autonomous search mission.
- uFldBeaconRangeSensor: Typically run in a shoreside community. Configured with one or more beacons with known beacon locations. Takes range requests from a remote vehicle and returns a range report indicating that vehicle's range to nearby beacons. Range requests may or may not be answered depending on range to beacon. Reports may have noise added and may or may not include beacon ID. See the documentation for uFldBeaconRangeSensor, part of the uField Toolbox.
- uFldContactRangeSensor: Typically run in a shoreside community. Takes reports from remote vehicles, notes their position. Takes a range request from a remote vehicle and returns a range report indicating that vehicle's range to nearby vehicles. Range requests may or may not be answered dependent on inter-vehicle range. Reports may also have noise added to their range values. See the documentation for uFldContactRangeSensor, part of the uField Toolbox.